Week 7

Task 1

```
class ParkingSystem {

    int big;
    int medium;
    int small;

    public ParkingSystem(int big, int medium, int small) {
        this.big = big;
        this.medium = medium;
        this.small = small;
    }

    public boolean addCar(int carType) {

        if (carType == 1) {
            if (big > 0) {
                big--;
                return true;
            }
        }

        else if (carType == 2) {
            if (medium > 0) {
                medium--;
                return true;
            }
```

```
        }

        else if (carType == 3) {
            if (small > 0) {
                small--;
                return true;
            }
        }

        return false;
    }
}
```

Task 3

```
import java.util.*;
class UndergroundSystem {
    HashMap<Integer, Pair> checkInMap;
    HashMap<String, Route> routeMap;
    public UndergroundSystem() {
        checkInMap = new HashMap<>();
        routeMap = new HashMap<>();
    }
    public void checkIn(int id, String stationName, int t) {
        checkInMap.put(id, new Pair(stationName, t));
    }
    public void checkOut(int id, String stationName, int t) {
        Pair p = checkInMap.get(id);
        checkInMap.remove(id);
```

```
        String startStation = p.station;
        int startTime = p.time;
        int travelTime = t - startTime;
        String route = startStation + "-" + stationName;
        Route r = routeMap.getOrDefault(route, new Route());
        r.totalTime += travelTime;
        r.count += 1;
        routeMap.put(route, r);
    }
    public double getAverageTime(String startStation, String endStation) {
        String route = startStation + "-" + endStation;
        Route r = routeMap.get(route);

        return (double) r.totalTime / r.count;
    }
}
class Pair {
    String station;
    int time;
    Pair(String station, int time) {
        this.station = station;
        this.time = time;
    }
}
class Route {
    int totalTime = 0;
    int count = 0;
}
```

Task 4

```
import java.util.*;
class BrowserHistory {
    List<String> history;
    int current;
    public BrowserHistory(String homepage) {
        history = new ArrayList<>();
        history.add(homepage);
        current = 0;
    }
    public void visit(String url) {
        while (history.size() > current + 1) {
            history.remove(history.size() - 1);
        }
        history.add(url);
        current++;
    }
    public String back(int steps) {
        current = Math.max(0, current - steps);
        return history.get(current);
    }
public String forward(int steps) {
        current = Math.min(history.size() - 1, current + steps);
        return history.get(current);
    }
}
```

Task -5

```
class MyHashSet {
    private boolean[] set;
    public MyHashSet() {
```

```
        set = new boolean[1000001];
    }
    public void add(int key) {
        set[key] = true;
    }
    public void remove(int key) {
        set[key] = false;
    }
    public boolean contains(int key) {
        return set[key];
    }
}
```

Task-6

```
import java.util.*;

class MyStack {

    private Queue<Integer> q1;
    private Queue<Integer> q2;

    public MyStack() {
        q1 = new LinkedList<>();
        q2 = new LinkedList<>();
    }

    public void push(int x) {
        q1.offer(x);
    }
```

```
public int pop() {
    while (q1.size() > 1) {
        q2.offer(q1.poll());
    }

    int top = q1.poll();
    Queue<Integer> temp = q1;
    q1 = q2;
    q2 = temp;

    return top;
}

public int top() {
    while (q1.size() > 1) {
        q2.offer(q1.poll());
    }

    int top = q1.peek();
    q2.offer(q1.poll());
    Queue<Integer> temp = q1;
    q1 = q2;
    q2 = temp;

    return top;
}

public boolean empty() {
    return q1.isEmpty();
```

```
    }
}
```

Task -7

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

class Animal{
    void walk(){
        System.out.println("I am walking");
    }
}

class Bird extends Animal{
    void fly(){
        System.out.println("I am flying");
    }
    void sing(){
        System.out.println("I am singing");
    }
}
public class Solution{

   public static void main(String args[]){

      Bird bird = new Bird();
```

```
    bird.walk();
    bird.fly();
```

Task -8

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;
class Arithmetic{
   int add(int a,int b){
      return a+b;
   }
}
class Adder extends Arithmetic{
}
//Write your code here

public class Solution{
   public static void main(String []args){
      // Create a new Adder object
      Adder a = new Adder();

      // Print the name of the superclass on a new line
      System.out.println("My superclass is: " + a.getClass().getSuperclass().getName());

      // Print the result of 3 calls to Adder's `add(int,int)` method as 3 space-separated
integers:
      System.out.print(a.add(10,32) + " " + a.add(10,3) + " " + a.add(10,10) + "\n");
    }
```

```
}
```

Task -9

```
import java.util.*;

class MyQueue {

    private Stack<Integer> inStack;
    private Stack<Integer> outStack;

    public MyQueue() {
        inStack = new Stack<>();
        outStack = new Stack<>();
    }

    public void push(int x) {
        inStack.push(x);
    }

    public int pop() {
        moveIfNeeded();
        return outStack.pop();
    }

    public int peek() {
        moveIfNeeded();
        return outStack.peek();
    }

    public boolean empty() {
```

```
        return inStack.isEmpty() && outStack.isEmpty();
    }

    private void moveIfNeeded() {
        if (outStack.isEmpty()) {
            while (!inStack.isEmpty()) {
                outStack.push(inStack.pop());
            }
        }
    }
}
```

Task -10

```
import java.util.*;
class MyStack {
    private Queue<Integer> q;
    public MyStack() {
        q = new LinkedList<>();
    }
    public void push(int x) {
        q.offer(x);
        for (int i = 0; i < q.size() - 1; i++) {
            q.offer(q.poll());
        }
    }
  public int pop() {
        return q.poll();
    }
    public int top() {
        return q.peek();
```

```
    }
    public boolean empty() {
        return q.isEmpty();
    }
}
```